

June 2026



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

Briscola RL

Training RL Agents on an Italian Card Game

Presented by Giacomo Serafini

INTRO I.

-Briscola-



- 2 players, 40-card Italian deck, 20 tricks per game
- Trump suit (**briscola**) revealed face-up at game start
- Trick winner decided by same suit → higher strength wins; trump beats non-trump; otherwise first player wins
- **Point cards:** Ace (11), 3 (10), King (4), Knight (3), Jack (2) → total 120 pts
- At the end, who has more than 60 points wins
- **RL challenges:** imperfect information, delayed terminal reward, stochastic opponent

INTRO II.

-DQN-

- Off-policy, value-based RL algorithm to approximate the Q-function using NNs
- Policy Network to predict action values $Q(s,a)$ using the Bellman update rule:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

- Target Network to stabilize training → every x steps, copy policy weights
- Experience Replay to store past experiences
- ϵ -greedy exploration to discover new strategies

INTRO III.

-PPO-

- On-policy, policy gradient RL algorithm to directly learn a policy $\pi(a | s)$
- **Actor-Critic architecture:** Actor selects actions, Critic evaluates states
- Optimizes the policy using **advantage estimates** instead of raw rewards:

$$A(s, a) = Q(s, a) - V(s)$$

→ GAE: $A_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$ where $\delta_t = R_t + \gamma V_{t+1} - V_t$

- **Clipped objective** to limit policy changes and improve stability

$$L = \min(r(\theta)A, \text{clip}(r(\theta), 1 - \varepsilon, 1 + \varepsilon)A) \quad \text{where } r(\theta) = \frac{\pi_{new}(a|s)}{\pi_{old}(a|s)}$$

- Collects trajectories and updates using on-policy learning → Rollout Buffer

ENVIRONMENT

- Custom *gymnasium.Env*: Briscola class
- **Action Space:** *Discrete(40)* $\rightarrow$ one action per card in the deck; masked to valid hand cards
- **Observation Space:** *Dict* $\rightarrow$ hand (40-bit), table card (40-bit), briscola suit (4-bit), played cards (40-bit), is_first (1-bit) = 125-dimensional input
- **Reward:** trick points (± 0.2 to ± 1.1) + terminal bonus (± 20) for win/loss
- **Action Masking:** hand encoding doubles as mask (invalid logits set to $-\infty$)

DQN TRAINING

- **Architecture:** MLP $125 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 40$, ReLU activations
- **Replay Buffer:** 100k, batch size 128 $\rightarrow$ random mini-batches of transitions are sampled for learning
- **ϵ -greedy policy:** ϵ decay: $0.9 \rightarrow 0.01$ multiplicative (per step, rate 0.9999)
- **Masking:** hand observation = action mask $\rightarrow$ invalid actions set to $-1e9$

I.

Trained for 200K episodes against random opponent

II.

Trained for 500K episodes with auto-play: every 20k episodes a snapshot was created and the opponent was chosen between “random” and “snapshot”

PPO TRAINING

- **Architecture:** two networks, Actor and Critic, both $125 \rightarrow 256 \rightarrow 256 \rightarrow 128$ then Actor: $128 \rightarrow 40$, while Critic $128 \rightarrow 1$, ReLU activations
- **Rollout Buffer:** experiences are collected on-policy and used for multiple update epochs with shuffled mini-batches (2048 rollout steps, 256 batch size, 5 epochs)
- **Generalized Advantage Estimation (GAE)** is used to compute advantages
- **Loss:** *policy loss* = clipped PPO objective; *value loss* = MSE between predicted values and GAE-based returns

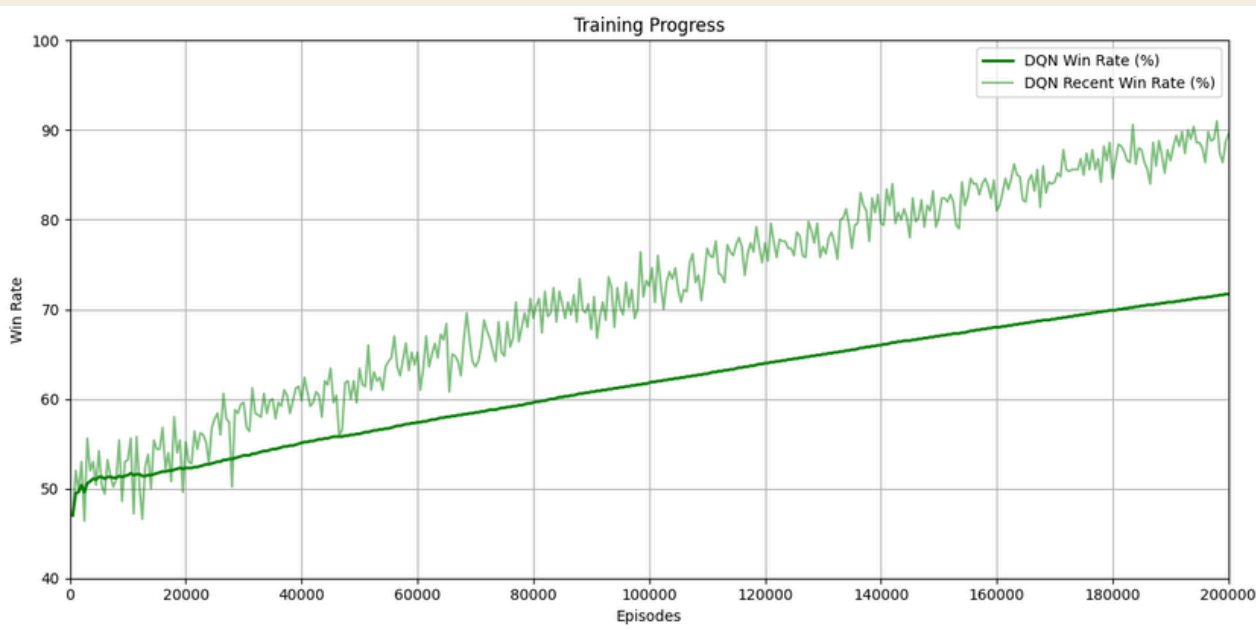
I.

Trained for 500K/1000K episodes against random opponent

II.

Trained for 1000K episodes with auto-play: every 40k episodes a snapshot was created and the opponent was chosen between “random” and “snapshot”

RESULTS



vs	RANDOM	DQN	PPO (500K)	PPO (1000K)	DQN (auto-500K)	PPO (auto-1000K)
DQN	W: 89.1% L: 10.2%	~50%	W: 58.7% L: 39.8%	W: 53.6% L: 44.5%	W: 66.8% L: 31.6%	W: 50.8% L: 47.6%
PPO	W: 87.5% L: 11.7%	W: 44.7% L: 53.3%	W: 53.8% L: 44.4%	~50%	W: 64.6% L: 33.7%	W: 47.5% L: 50.7%

- DQN appears more sample-efficient and faster to converge in this setup
- PPO appears more stable under changing dynamics but slower to improve
[tested on 10K episodes]

LIMITATIONS

- Training is computationally expensive and requires a large number of episodes for stable convergence
- The environment is a custom Briscola simulation → results are not directly comparable to standard RL benchmarks
- Observations don't include known played cards strategically weighted

FUTURE WORK

- Auto-play training needs further work to improve robustness against different opponent strategies
- More extensive hyper-parameter tuning

June 2026



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

THANK YOU!

Code: <https://github.com/JackSerafini/BriscolaRL>

Presented by Giacomo Serafini